We've indexed our first document for Elasticsearch! Indexing a document is similar to inserting a record in a relational database.

CONSTITUENTS OF THE REQUEST

The request we sent (`PUT books/_doc/1`) needs a bit of distilling. There are five parts to this request, and we quickly go over them here (if you wish, you can skip this section for now and come back to it once you've read the rest of the chapter):

- `PUT` *method*—`PUT` is an HTTP verb (also known as a *method*) that indicates we are sending a request to the server to create a resource (a `book` document, in this case). Elasticsearch uses the HTTP protocol for its RESTful API invocations, so we expect `PUT`, `POST`, `GET`, `DELETE`, and other standard HTTP methods in the request URL syntax.
- `books` *index*—The `books` part of our URL is called an *index:* a bucket for collecting all `book` documents. It is similar to a table in a relational database. Only `book` documents are stored in this `books` index (although `books` index is expected to hold only book documents, in theory, there's nothing stopping us indexing other types, too—it is our responsibility to not mix up types).
- `_doc` *endpoint*—The endpoint is a constant part of the path that's associated with the operation being performed. In earlier versions of Elasticsearch (version < 7.0), `_doc`'s place used to be filled by a document's mapping type. The mapping types were deprecated, and `_doc` replaced them as a generic constant endpoint path in the URL (see the sidebar "Document types and the _doc endpoint" for more).
- *Document ID*—The number `1` in our URL represents the document's ID. It is like a primary key for a record in a database. We use this identifier to retrieve the document.
- *Request body*—The body of the request is the JSON representation of the book data. Elasticsearch expects all data to be sent as JSON-formatted documents. It also supports nested objects sent in JSON's nested objects format.

Document types and the _doc endpoint

Prior to the 7.x version of Elasticsearch, an index could hold multiple types of entities (for example, a `books` index could hold not just books but also book reviews, book sales, bookshops, and so on). Having all types of documents in a single index led to complications. Field mappings were shared across multiple types, leading to errors as well as data sparsity. To avoid problems with types and their management, Elastic decided to remove the types.

In earlier versions, the invocation URL with a type looked like this: <*index_name*>/<*type*>/<*id*> (for example, books/book/1). The types were deprecated in version 7.x. Now an index is expected to be dedicated to one and only one type: `_doc` is an endpoint etched into the URL. We learn about the removal of types in chapter 5.

The gist is that we used an HTTP `PUT` method to index a document with an ID of `1` into the `books` index for Elasticsearch. When indexing our first document, did you